

1. LAB 1

Program

```
#include <stdio.h>
int main() {
    char *str = "Hello World";
    int i = 0;
    printf("Original string: %s\n", str);
    printf("After XOR with 0: ");
    while(str[i] != '\0') {
        char result = str[i] ^ 0;
        printf("%c", result);
        i++;
    }
    printf("\n");
    return 0;
}
```

Sample output

```
Original string: Hello World
After XOR with 0: Hello World
```

2. LAB 2

Program

```
#include <stdio.h>
int main() {
    char *str = "Hello World";
    printf("Original string: %s\n", str);
    // Using AND with 127
    printf("After AND with 127:\n");
    for (int i = 0; str[i] != '\0'; i++) {
        char result = str[i] & 127;
        printf("%c", result);
    }
    printf("\n\n");
    // Using XOR with 127
    printf("After XOR with 127:\n");
    for (int i = 0; str[i] != '\0'; i++) {
        char result = str[i] ^ 127;
        printf("%c", result);
    }
    printf("\n");
    return 0;
}
```

Sample output

```
Original string: Hello World
After AND with 127:
Hello World
```

```
After XOR with 127:
7\x1A\x13\x13\x10_{\x10\x0D\x13\x1B
```

3. LAB 3(a) - Caesar Cipher

Program

```
import java.util.Scanner;
public class SimpleCaesar {
    // Shift one character
    static char shiftChar(char ch, int shift) {
        if (ch >= 'A' && ch <= 'Z') {
            return (char) ('A' + (ch - 'A' + shift) % 26);
        }
        if (ch >= 'a' && ch <= 'z') {
            return (char) ('a' + (ch - 'a' + shift) % 26);
        }
        return ch; // don't change spaces, numbers, symbols
    }
    // Encrypt
    static String encrypt(String text, int shift) {
        StringBuilder sb = new StringBuilder();
        for (char ch : text.toCharArray()) {
            sb.append(shiftChar(ch, shift));
        }
        return sb.toString();
    }
    // Decrypt
    static String decrypt(String text, int shift) {
        return encrypt(text, 26 - shift); // just shift backward
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter text: ");
        String text = sc.nextLine();
        System.out.print("Enter shift: ");
        int shift = sc.nextInt();
        System.out.println("Encrypted: " + encrypt(text, shift));
        System.out.println("Decrypted: " + decrypt(encrypt(text, shift), shift));
        sc.close();
    }
}
```

Sample output

```
Enter text: HELLO WORLD
Enter shift: 3
Encrypted: KH00R ZRU0G
Decrypted: HELLO WORLD
```

4. LAB 3(b) - Substitution Cipher

Program

```
import java.util.Scanner;
public class SubstitutionCipher {
    // Standard alphabet
    static final String ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    // Substitution key - Must be 26 unique letters
    static final String KEY = "QWERTYUIOPASDFGHJKLZXCVBNM";
    public static String encrypt(String plainText) {
        plainText = plainText.toUpperCase();
        StringBuilder cipherText = new StringBuilder();
        for (int i = 0; i < plainText.length(); i++) {
            char c = plainText.charAt(i);
            if (Character.isLetter(c)) {
                int index = ALPHABET.indexOf(c);
                char encryptedChar = KEY.charAt(index);
                cipherText.append(encryptedChar);
            } else {
                // Keep spaces and symbols as it is
                cipherText.append(c);
            }
        }
        return cipherText.toString();
    }
    // Decryption function
    public static String decrypt(String cipherText) {
        cipherText = cipherText.toUpperCase();
        StringBuilder plainText = new StringBuilder();
        for (int i = 0; i < cipherText.length(); i++) {
            char c = cipherText.charAt(i);
            if (Character.isLetter(c)) {
                int index = KEY.indexOf(c);
                char decryptedChar = ALPHABET.charAt(index);
                plainText.append(decryptedChar);
            } else {
                plainText.append(c);
            }
        }
        return plainText.toString();
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("--- Substitution Cipher ---");
        System.out.println("Alphabet : " + ALPHABET);
        System.out.println("Key : " + KEY);
        System.out.print("\nEnter Plain Text: ");
        String plain = sc.nextLine();
        String encrypted = encrypt(plain);
        System.out.println("Encrypted Text: " + encrypted);
        String decrypted = decrypt(encrypted);
        System.out.println("Decrypted Text: " + decrypted);
        sc.close();
    }
}
```

Sample output

```
--- Substitution Cipher ---
Alphabet : ABCDEFGHIJKLMNOPQRSTUVWXYZ
Key : QWERTYUIOPASDFGHJKLZXCVBNM

Enter Plain Text: HELLO WORLD
Encrypted Text: ITFFG VGJFE
Decrypted Text: HELLO WORLD
```

5. LAB 3(c) - Hill Cipher

Program

```
import java.util.Scanner;
class HillRow {
static String encrypt(String text, int[][] key) {
text = text.toUpperCase().replaceAll(" ", "");
if (text.length() % 2 != 0) text += "X";
String cipher = "";
for (int i = 0; i < text.length(); i += 2) {
int p1 = text.charAt(i) - 'A';
int p2 = text.charAt(i + 1) - 'A';
// Row vector * Matrix: C = P * K
// C1 = P1*K00 + P2*K10
// C2 = P1*K01 + P2*K11
int c1 = (p1 * key[0][0] + p2 * key[1][0]) % 26;
int c2 = (p1 * key[0][1] + p2 * key[1][1]) % 26;
cipher += (char) (c1 + 'A');
cipher += (char) (c2 + 'A');
}
return cipher;
}
static String decrypt(String cipher, int[][] key) {
// Inverse for ROW method
int det = (key[0][0] * key[1][1] - key[0][1] * key[1][0]) % 26;
if (det < 0) det += 26;
int detInv = -1;
for (int i = 0; i < 26; i++) {
if ((det * i) % 26 == 1) { detInv = i; break; }
}
if (detInv == -1) return "Key not invertible";
int[][] invKey = new int[2][2];
invKey[0][0] = (key[1][1] * detInv) % 26;
invKey[0][1] = (-key[0][1] * detInv) % 26;
invKey[1][0] = (-key[1][0] * detInv) % 26;
invKey[1][1] = (key[0][0] * detInv) % 26;
for (int i = 0; i < 2; i++)
for (int j = 0; j < 2; j++)
if (invKey[i][j] < 0) invKey[i][j] += 26;
// Decrypt: P = C * K_inverse
String plain = "";
for (int i = 0; i < cipher.length(); i += 2) {
int c1 = cipher.charAt(i) - 'A';
int c2 = cipher.charAt(i + 1) - 'A';
int p1 = (c1 * invKey[0][0] + c2 * invKey[1][0]) % 26;
int p2 = (c1 * invKey[0][1] + c2 * invKey[1][1]) % 26;
plain += (char) (p1 + 'A');
plain += (char) (p2 + 'A');
}
return plain;
}
}
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
// Key [[3,3],[2,5]]
int[][] key = {{3, 3}, {2, 5}};
System.out.print("Enter plaintext: ");
String plain = sc.nextLine();
String enc = encrypt(plain, key);
System.out.println("Encrypted: " + enc);
String dec = decrypt(enc, key);
System.out.println("Decrypted: " + dec);
}
}
```

Sample output

```
Enter plaintext: HELP
Encrypted: DPLE
Decrypted: HELP
```

6. LAB 4 - Rail Fence Transposition

Program

```
import java.util.*;
class RailFenceCipher {
    static String encrypt(String t, int r){
        StringBuilder[] a = new StringBuilder[r];
        for(int i=0;i<r;i++) a[i]=new StringBuilder();
        int row=0,d=1;
        for(char c:t.toCharArray()){
            a[row].append(c);
            if(row==0) d=1; else if(row==r-1) d=-1;
            row+=d;
        }
        StringBuilder s=new StringBuilder();
        for(var x:a) s.append(x);
        return s.toString();
    }
    static String decrypt(String c, int r){
        int n=c.length(); char[][] f=new char[r][n];
        int row=0,d=1;
        for(int i=0;i<n;i++){
            { f[row][i]='*'; if(row==0) d=1; else if(row==r-1) d=-1; row+=d; }
            int k=0;
            for(int i=0;i<r;i++) for(int j=0;j<n;j++) if(f[i][j]=='*') f[i][j]=c.charAt(k++);
            StringBuilder s=new StringBuilder(); row=0; d=1;
            for(int i=0;i<n;i++){
                { s.append(f[row][i]); if(row==0) d=1; else if(row==r-1) d=-1; row+=d; }
            }
            return s.toString();
        }
    }
    public static void main(String[] a){
        Scanner sc=new Scanner(System.in);
        System.out.print("Text: "); String t=sc.nextLine();
        System.out.print("Rails: "); int r=sc.nextInt();
        String e=encrypt(t,r);
        System.out.println("Encrypted: "+e);
        System.out.println("Decrypted: "+decrypt(e,r));
    }
}
```

Sample output

```
Text: WEAREDISCOVEREDFLEEATONCE
Rails: 3
Encrypted: WECRLTEERDSOEEFEAOCAIVDEN
Decrypted: WEAREDISCOVEREDFLEEATONCE
```

7. LAB 5 - Single and Double Transposition

Program

```
import java.util.*;
public class ColumnarTranspositionCipher {
    static int[] getKey(String keykey) {
        String k = keykey.toUpperCase();
        int n = k.length();
        int[] key = new int[n];
        Integer[] idx = new Integer[n];
        for (int i = 0; i < n; i++) idx[i] = i;
        Arrays.sort(idx, (a, b) -> k.charAt(a) != k.charAt(b)? k.charAt(a) - k.charAt(b) : a
        - b);
        for (int r = 0; r < n; r++) key[idx[r]] = r + 1;
        return key;
    }
    static String encrypt(String pt, int[] key) {
        int c = key.length, r = (int) Math.ceil((double) pt.length() / c);
        char[][] g = new char[r][c];
        for (int i = 0; i < r * c; i++) g[i / c][i % c] = i < pt.length()? pt.charAt(i) : '_';
        StringBuilder sb = new StringBuilder();
        for (int k = 1; k <= c; k++)
            for (int j = 0; j < c; j++) if (key[j] == k)
                for (int i = 0; i < r; i++) sb.append(g[i][j]);
        return sb.toString();
    }
    static String decrypt(String ct, int[] key) {
        int c = key.length, r = ct.length() / c;
        char[][] g = new char[r][c];
        int p = 0;
        for (int k = 1; k <= c; k++)
            for (int j = 0; j < c; j++) if (key[j] == k)
                for (int i = 0; i < r; i++) g[i][j] = ct.charAt(p++);
        StringBuilder sb = new StringBuilder();
        for (char[] row : g) for (char ch : row) sb.append(ch);
        return sb.toString();
    }
    // Double Transposition
    static String doubleEncrypt(String pt, int[] key) {
        return encrypt(encrypt(pt, key), key);
    }
    static String doubleDecrypt(String ct, int[] key) {
        return decrypt(decrypt(ct, key), key);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Plaintext: ");
        String msg = sc.nextLine().replace(" ", "");
        System.out.print("Key: ");
        String skey = sc.next();
        int[] key = getKey(skey);
        System.out.println("\nNumeric Key: " + Arrays.toString(key));
        String singleEnc = encrypt(msg, key);
        System.out.println("\n--- Single Transposition ---");
        System.out.println("Encrypted: " + singleEnc);
        System.out.println("Decrypted: " + decrypt(singleEnc, key));
        String doubleEnc = doubleEncrypt(msg, key);
        System.out.println("\n--- Double Transposition ---");
        System.out.println("Encrypted: " + doubleEnc);
        System.out.println("Decrypted: " + doubleDecrypt(doubleEnc,
        key).replace("_", ""));
    }
}
```

Sample output

Plaintext: WEAREDISCOVEREDFLEEATONCE
Key: ZEBRA

Numeric Key: [5, 3, 2, 4, 1]
--- Single Transposition ---

Encrypted: EODAEASRENEIELORCEECWDVFT
Decrypted: WEAREDISCOVEREDFLEEATONCE

--- Double Transposition ---
Encrypted: ENOCTDREEVOSICDAELEFEAERW
Decrypted: WEAREDISCOVEREDFLEEATONCE

8. LAB 6 - DES

Program

```
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.DESKeySpec;
import java.util.Base64;
import java.util.Scanner;
public class DESWithUserKey {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Plaintext: ");
        String plainText = sc.nextLine();
        System.out.print("Enter Key (must be 8 chars e.g. 12345678): ");
        String keyInput = sc.nextLine();
        // 1. Make key from user input string
        DESKeySpec desKeySpec = new DESKeySpec(keyInput.getBytes());
        SecretKeyFactory keyFactory = SecretKeyFactory.getInstance("DES");
        SecretKey secretKey = keyFactory.generateSecret(desKeySpec);
        // 2. Encrypt
        Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");
        cipher.init(Cipher.ENCRYPT_MODE, secretKey);
        byte[] encryptedBytes = cipher.doFinal(plainText.getBytes());
        String encryptedText =
            Base64.getEncoder().encodeToString(encryptedBytes);
        System.out.println("Encrypted: " + encryptedText);
        // 3. Decrypt
        cipher.init(Cipher.DECRYPT_MODE, secretKey);
        byte[] decryptedBytes =
            cipher.doFinal(Base64.getDecoder().decode(encryptedText));
        String decryptedText = new String(decryptedBytes);
        System.out.println("Decrypted: " + decryptedText);
    }
}
```

Sample output

```
Enter Plaintext: HELLO WORLD
Enter Key (must be 8 chars e.g. 12345678): 12345678
Encrypted: +dhTwlyrdA35S/6Xk5R+yQ==
Decrypted: HELLO WORLD
```